

CommodityGH

AWS Service Selection & Architecture Justification

1. Current System Architecture (As-Built)

The following is a precise summary of the system as it currently exists, derived directly from the codebase. All AWS recommendations in subsequent sections map to these specific components.

Layer	Technology	Key Details
Backend	Spring Boot 3.5.11 (Java 21)	Stateless REST API with 13 controllers: Auth, PriceRecord, Analytics, DataExport, MarketHealth, SeasonalPattern, and more.
Database	PostgreSQL	10 JPA entities (User, Commodity, Market, City, PriceRecord, MarketHealthScore, SeasonalPattern, ExportLog, PriceRecordAudit, PasswordResetToken) managed by Flyway with 14 migrations.
Authentication	Custom JWT (jjwt 0.12.3)	JwtTokenProvider + JwtAuthenticationFilter, BCrypt password hashing, @EnableMethodSecurity, stateless sessions — no external identity provider.
Frontend	React 18 + Vite + TypeScript	SPA using react-router-dom and Axios; deployed as a static dist/ bundle. TailwindCSS + shadcn/ui for UI; Recharts for analytics visualisations.
Email	Brevo SMTP (port 2525)	JavaMailSender + Thymeleaf templates for password reset, agent approval/rejection, and admin notifications. @Async dispatch.
Scheduled Jobs	Spring @Scheduled	Esoko price scraper (daily 5 AM), market health scores (daily 2 AM), seasonal patterns (monthly), SLA breach check (hourly), cache eviction (15 min), export log cleanup (weekly), token cleanup (nightly).
Caching	In-memory ConcurrentMapCache	Single dashboardSummary cache, evicted every 15 minutes by the scheduler.
Data Export	Apache POI + CSV	Up to 100k rows generated in-memory as byte[], streamed directly to the client.
Web Scraping	Jsoup	Esoko marketplace HTML scraping; results persisted as PriceRecord entities.
Containerisation	Docker	Multi-stage build: eclipse-temurin:21-jdk-alpine → eclipse-temurin:21-jre-alpine, exposed on port 8080.
Current Hosting	Render.com	Backend deployed as a Docker container; frontend on Render Static Sites or equivalent.
Connection Pool	HikariCP	max-pool-size: 5, min-idle: 1 — tuned for Render's idle timeout behaviour.

2. Recommended AWS Services

Each recommendation maps directly to an existing component in the codebase. No new frameworks, no code refactoring, and no architectural redesign are required.

2.1 Compute — AWS App Runner

Replaces: Render Docker deployment

Codebase entry point: Dockerfile → java -jar app.jar on port 8080

Aspect	Detail
Why App Runner	Accepts a Docker image (or source code), auto-builds, auto-deploys, and auto-scales — exactly how Render works today. The existing Dockerfile works as-is with zero code changes.
How It Fits	Point App Runner to the ECR image (or GitHub repo). It detects port 8080 from EXPOSE. Set environment variables (DB_URL, JWT_SECRET, BREVO_SMTP_PASSWORD, etc.) in the App Runner console. Spring Boot starts, Flyway runs migrations, and schedulers fire — identical to the current setup.
Scaling	Auto-scales from 1 to N instances based on concurrent requests. Scales to zero on the Paused tier to reduce cost during idle hours.

Alternatives Considered & Rejected

Alternative	Why Rejected
ECS Fargate	Requires Task Definitions, Services, Target Groups, ALB, and VPC networking — significant operational overhead for a single-container application. App Runner abstracts all of this.
EC2	Requires OS patching, instance sizing, and ASG + ALB configuration. The operational burden is unjustifiable for this scope.
Elastic Beanstalk	Legacy-feeling with slower deployments and less predictable scaling. App Runner is the modern replacement for simple container workloads.
Lambda	The backend is a long-running Spring Boot process with @Scheduled cron jobs, in-memory caching, and HikariCP pooling — all incompatible with Lambda's stateless, 15-minute-max execution model.

Estimated Monthly Cost	~\$7–15 / month
------------------------	-----------------

2.2 Database — Amazon RDS for PostgreSQL

Replaces: Render PostgreSQL (or external managed Postgres)

Connection: `_${DB_URL}` → standard JDBC URL via HikariCP pool

Aspect	Detail
Why RDS PostgreSQL	Drop-in replacement. Changing one environment variable (DB_URL to the RDS endpoint) is all that is required. Flyway runs the 14 existing migrations on startup; HikariCP connects; the PostgreSQLDialect stays unchanged.
What You Get	Automated daily backups (35-day retention), Multi-AZ failover, automated minor version upgrades, CloudWatch monitoring, and point-in-time recovery.
Instance Sizing	db.t4g.micro (2 vCPU, 1 GB RAM, Free Tier eligible) is sufficient for the current HikariCP settings of max-pool-size: 5 and min-idle: 1. Upgrade to db.t4g.small when traffic grows.

Alternatives Considered & Rejected

Alternative	Why Rejected
Aurora Serverless v2	Minimum cost is ~\$43/month (0.5 ACU always-on). For a project currently running on a free-tier database, this is over-provisioned. Aurora becomes appropriate when scaling beyond db.t4g.medium.
DynamoDB	The entire data model is relational: JPA entities with @ManyToOne relationships, Criteria API queries with joins, and Flyway SQL migrations. Migrating would require rewriting every repository, service, and query — a complete data-layer refactoring.
Aurora Provisioned	Same engine as RDS PostgreSQL but 2–3× the cost for the same instance class. Not justified until read replicas or global databases are required.

Estimated Monthly Cost

**\$0 (Free Tier, 12 months) →
~\$13 / month**

2.3 Frontend Hosting — Amazon S3 + CloudFront

Replaces: Current frontend hosting (Render Static Sites or similar)

Current build output: `vite build` → `dist/` folder (static HTML/JS/CSS)

Aspect	Detail
Why S3 + CloudFront	The React SPA builds to a static <code>dist/</code> directory. Uploading to S3 and fronting it with CloudFront delivers globally distributed, HTTPS-secured, cached content at the edge — with no server required.
How It Fits	<code>npm run build</code> → upload <code>dist/</code> to S3 → CloudFront invalidation. Setting <code>VITE_API_BASE_URL</code> to the App Runner backend URL at build time is all that is needed. The existing <code>axios.ts</code> base URL config (<code>import.meta.env.VITE_API_BASE_URL</code>) works unchanged.
SPA Routing	CloudFront is configured to return <code>index.html</code> for 403/404 errors, enabling react-router-dom client-side routing.

Alternatives Considered & Rejected

Alternative	Why Rejected
AWS Amplify Hosting	A valid option, but it adds abstraction that is not needed. S3 + CloudFront provides direct control over caching rules, headers, and costs. Amplify's per-build pricing also accumulates with frequent deploys.
App Runner (static files)	A waste of compute resources. S3 costs pennies per month versus \$7+/month for an always-on container serving static files.
EC2 + Nginx	Server management overhead for serving static files. Completely unnecessary at this scale.

Estimated Monthly Cost	~\$1–3 / month
------------------------	----------------

2.4 Email — Amazon SES (Simple Email Service)

Replaces: Brevo SMTP relay (smtp-relay.brevo.com:2525)

Current code: JavaMailSender in EmailService.java, configured via spring.mail. properties*

Aspect	Detail
Why SES	EmailService uses standard JavaMailSender (SMTP). SES provides an SMTP endpoint — changing four properties in application.yaml (host, port, username, password) is all that is required. Zero code changes.
Config Change	spring.mail.host: email-smtp.eu-west-1.amazonaws.com, port: 587, username/password: SES SMTP credentials. Thymeleaf templates, @Async dispatch, and MimeMessageHelper all remain unchanged.

Alternatives Considered & Rejected

Alternative	Why Rejected
Keep Brevo	Viable, but introduces an external dependency outside AWS. SES is cheaper (\$0.10 per 1,000 emails), natively integrated with IAM for credential management, and keeps all infrastructure within one provider.
Amazon SNS	SNS is for push notifications and pub/sub, not transactional HTML email. Thymeleaf-rendered HTML emails require SMTP — that is SES.
Amazon Pinpoint	Designed for marketing campaigns and analytics. Overkill for five transactional email types.

Estimated Monthly Cost	\$0 (62,000 free emails/month from App Runner)
------------------------	--

2.5 Secrets Management — AWS SSM Parameter Store (SecureString)

Replaces: Environment variables passed via Render dashboard

Current secrets: DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, BREVO_SMTP_PASSWORD, ADMIN_USERNAME, ADMIN_PASSWORD, ADMIN_EMAIL

Aspect	Detail
Why Parameter Store	Secrets are currently plain environment variables. Parameter Store stores them encrypted (AWS-managed KMS key), provides version history, and integrates natively with App Runner's 'reference secrets' feature. No code changes — App Runner injects them as environment variables at container start.
How It Fits	Parameters are created as /commoditygh/prod/DB_URL etc. (SecureString). In the App Runner service config, each env var is mapped to the SSM parameter ARN. Spring Boot reads them as \${DB_URL} exactly as before.

Alternatives Considered & Rejected

Alternative	Why Rejected
AWS Secrets Manager	\$0.40/secret/month + \$0.05 per 10k API calls. For 8 secrets that rarely rotate, that is \$3.20/month with no tangible benefit. Parameter Store SecureString is free (standard tier, up to 10,000 parameters).
Hardcoded in Config	Security anti-pattern. Secrets in source control are a significant liability.
HashiCorp Vault	Massive operational overhead (self-hosted). Completely unjustifiable for 8 secrets.

Estimated Monthly Cost	\$0 / month
------------------------	-------------

2.6 Container Registry — Amazon ECR (Elastic Container Registry)

Replaces: Render's built-in Docker image builds

Dockerfile: Multi-stage build producing app.jar on eclipse-temurin:21-jre-alpine

Aspect	Detail
Why ECR	A registry is required to store the Docker image for App Runner to pull from. ECR is the native AWS registry. A CI push triggers an automatic App Runner deployment.
How It Fits	docker build -t commoditygh . → docker push <ecr-repo-url>. The existing Dockerfile is unchanged.

Alternatives Considered & Rejected

Alternative	Why Rejected
Docker Hub	Free tier has rate limits (100 pulls per 6 hours for anonymous users). ECR has no pull limits within AWS, and pulling from Docker Hub to App Runner adds cross-network latency.
GitHub Container Registry (GHCR)	Viable, but App Runner has native ECR integration with auto-deploy on push. GHCR requires manual webhook setup.

Estimated Monthly Cost	~\$0.50–1 / month
------------------------	-------------------

2.7 CI/CD — GitHub Actions + ECR Push

<i>Extends: Current manual deploy workflow on Render</i>
--

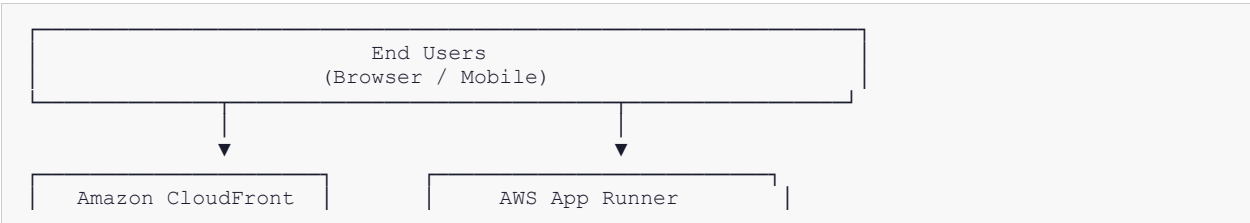
Aspect	Detail
Why GitHub Actions	The repositories (mawule-gabriel/commoditygh, TECHWITHKAY/FINAL-YEAR-PROJECT-) are already on GitHub. GitHub Actions is free for public repos and provides docker/build-push-action for ECR with no additional AWS service required.
Pipeline	git push → GitHub Actions runs mvn package -DskipTests → docker build → docker push to ECR → App Runner auto-deploys. Frontend: npm run build → aws s3 sync dist/ s3://bucket → aws cloudfront create-invalidation.

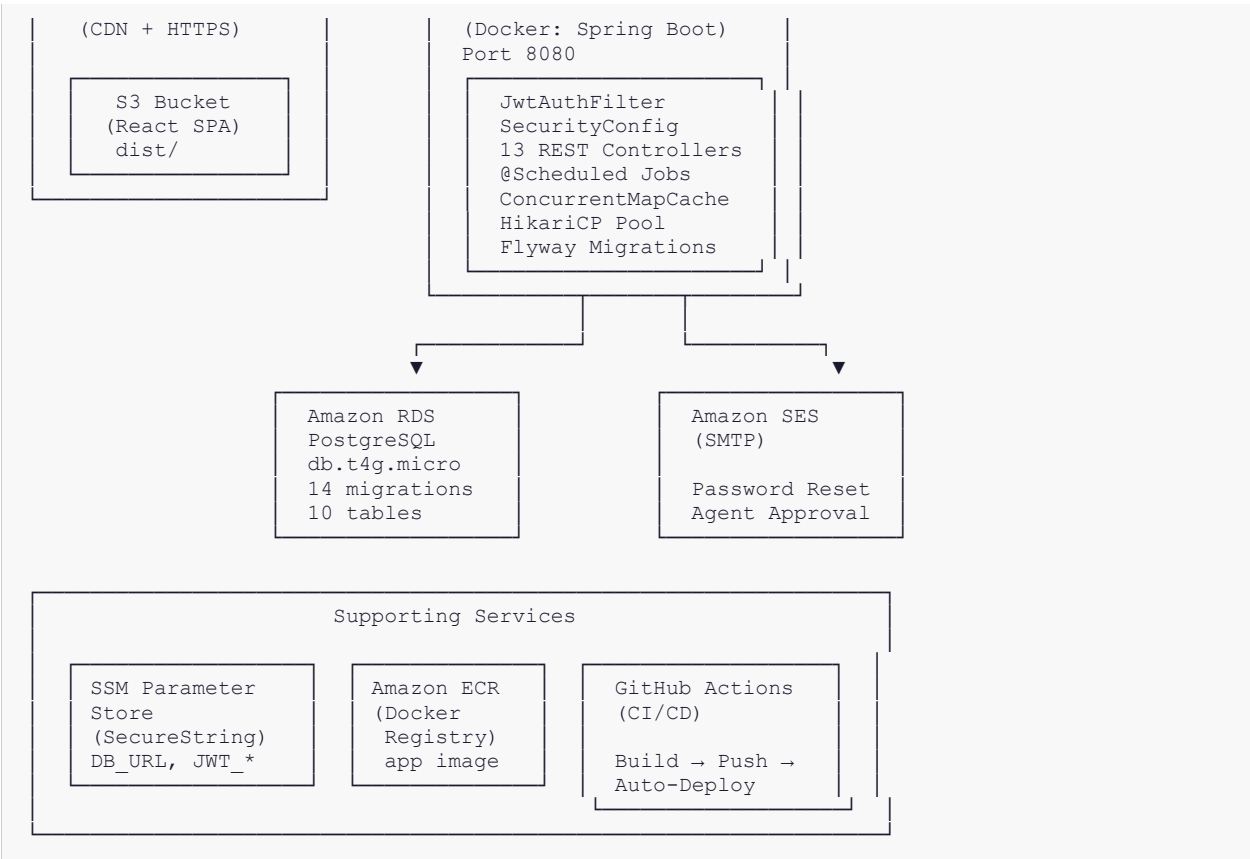
Alternatives Considered & Rejected

Alternative	Why Rejected
AWS CodePipeline + CodeBuild	Adds two new AWS services to learn and maintain. CodeBuild charges per build minute. GitHub Actions is already integrated with the repos and is free for public repos.
AWS CodeDeploy	Designed for EC2/on-prem deployments. App Runner handles deployment natively.

Estimated Monthly Cost	\$0 / month (public repos)
------------------------	----------------------------

3. Proposed AWS Architecture Diagram





4. Monthly Cost Estimate

Service	Tier / Size	Estimated Cost
App Runner	1 vCPU, 2 GB RAM, auto-pause	\$7–15
RDS PostgreSQL	db.t4g.micro, 20 GB gp3	\$0 (Free Tier) → \$13
S3	<100 MB static files	<\$0.10
CloudFront	<10 GB transfer/month	\$0 (Free Tier: 1 TB)
SES	<1,000 emails/month	\$0 (Free Tier: 62k/month)
ECR	~200 MB image	<\$0.50
SSM Parameter Store	8 parameters (standard)	\$0
GitHub Actions	Public repos	\$0
TOTAL		~\$8–30 / month

This estimate is significantly cheaper than most managed PaaS providers at equivalent capability.

Included at this price: automated backups, global CDN, HTTPS, auto-scaling, and secrets management.

5. When to Revisit This Architecture

The current architecture is intentionally right-sized. The following table outlines future scaling triggers and the corresponding upgrade paths.

Trigger	Recommended Upgrade
Multiple App Runner instances lose shared cache	Add ElastiCache (Redis) for the dashboardSummary cache.
Database exceeds db.t4g.small capacity	Upgrade to Aurora Serverless v2 for auto-scaling ACUs.
Need background job isolation (schedulers)	Extract @Scheduled tasks into ECS Scheduled Tasks or EventBridge + Lambda.
Traffic exceeds 10k requests/day or DDoS risk	Add AWS WAF on CloudFront + App Runner.
Need centralised logging and monitoring	CloudWatch Logs agent (App Runner already sends logs to CloudWatch by default).
User auth complexity grows (SSO, MFA, federation)	Evaluate migrating from custom JWT to Amazon Cognito.

6. Conclusion

This AWS architecture is a lift-and-shift from Render with zero code changes required. Every recommended service maps 1:1 to an existing component:

AWS Service	Maps To
App Runner	Existing Dockerfile + Spring Boot container
RDS PostgreSQL	JDBC DB_URL + Flyway + JPA entities
S3 + CloudFront	Vite dist/ build output
SES	JavaMailSender SMTP configuration
SSM Parameter Store	All current environment variables
ECR	Existing Docker image
GitHub Actions	Existing GitHub repositories

No unnecessary services. No refactoring. No over-engineering.

A production-grade AWS deployment at ~\$8–30 / month.